Homework 6

Comments/docstrings are required for problems 1 and 3. They are not required for problems 2.

Problem 0

It is highly recommended that you work with your group to fully complete the recent Discussion assignments related to the Palmer Penguins data set, as these will directly help with your project.

Problem 1

Gapminder is a foundation, based on Sweden, that aims to enhance basic awareness of basic facts about the socioeconomic global world. As part of their efforts, they collect detailed statistics on life expectancy, population, and GDP, sometimes going back over many years.

Here, we'll work with an excerpt of the Gapminder data. This excerpt has been packaged up and made available via Jenny Bryan's `gapminder` repo on Github.

Run the code below to retrieve the data and take a look. As usual, you can also directly download the data by pasting the url into your browswer, saving the file, and reading it in locally via `pandas.read_csv`.

In [586…

```
import pandas as pd

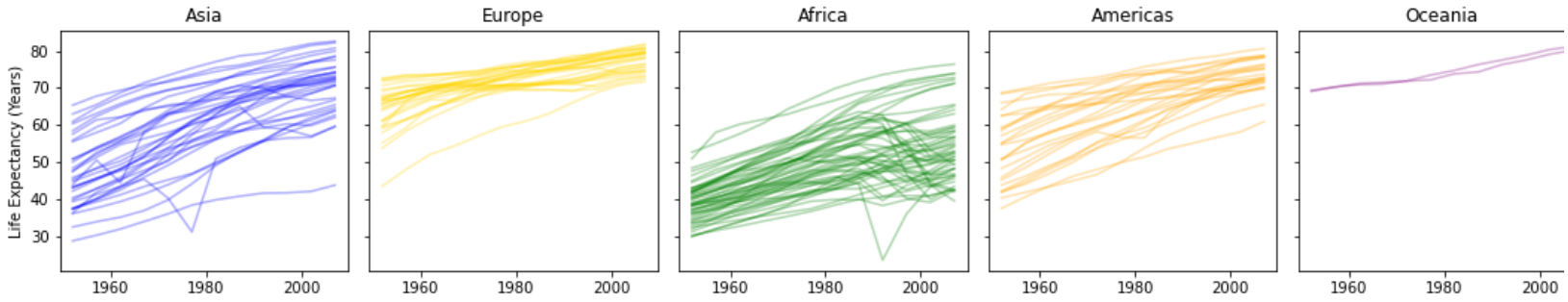
url = "https://philchodrow.github.io/PIC16A/datasets/gapminder.csv"
gapminder = pd.read_csv(url)
gapminder
```

Out[586]:

	country	continent	year	lifeExp	pop	gdpPercap
0	Afghanistan	Asia	1952	28.801	8425333	779.445314
1	Afghanistan	Asia	1957	30.332	9240934	820.853030
2	Afghanistan	Asia	1962	31.997	10267083	853.100710
3	Afghanistan	Asia	1967	34.020	11537966	836.197138
4	Afghanistan	Asia	1972	36.088	13079460	739.981106
...	...	...	...	...	...	...
1699	Zimbabwe	Africa	1987	62.351	9216418	706.157306
1700	Zimbabwe	Africa	1992	60.377	10704340	693.420786
1701	Zimbabwe	Africa	1997	46.809	11404948	792.449960
1702	Zimbabwe	Africa	2002	39.989	11926563	672.038623
1703	Zimbabwe	Africa	2007	43.487	12311143	469.709298

1704 rows × 6 columns

Use the `gapminder` data to create the folowing visualization. Here, each trendline corresponds to a distinct country, which in turn lies on the stated continent.



You should achieve this result **without for-loops**, and also without manually creating the plot on each axis. You may find it useful to define additional data structures, such as dictionaries, that assign colors or axis indices to each continent. Feel free to modify aesthetic details of the plots, such as the colors.

Hint: Try `df.groupby(["country"]).apply(f)`, where `f` is some very smart function that you have figured out.

In [587…

```
# your solution here

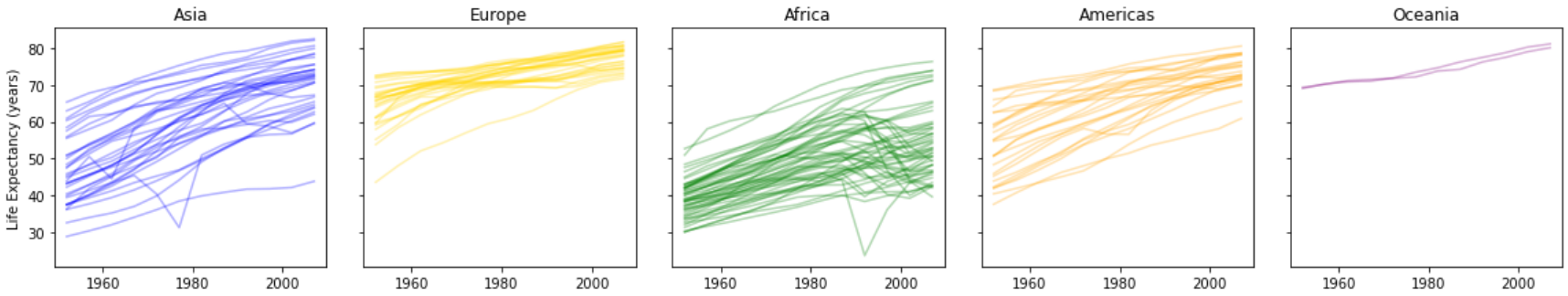
# creating a dict that matches colors to each continent, to use later
colors = {"Asia":"Blue","Europe":"Gold","Africa":"Green","Americas":"Orange","Oceania":"Purple"}
# creating a dict that matches a number to each continent, to use later for subplot assignment
numbers = {"Asia":0,"Europe":1,"Africa":2,"Americas":3,"Oceania":4}
# creating a wide plot with subplots in 1 row and 5 columns, that all share y-axes
fig,ax=plt.subplots(1,5,figsize=(16,3),sharey=True)

def f(df, **kwargs):
    """
    A function that graphs life expectancy vs years by continent.
    Each continent is on its own subplot with its own distinct color.
    parameter 1, df: the dataframe whose data is being graphed
    Also accepts kwargs, any arbitrary number of keyword arguments as specified by the user.
    """
    continent=df["continent"].iloc[0] #getting the correct continent for each country in the df
    ax[numbers[continent]].plot(df["year"],df["lifeExp"], # on correct subplot for that continent (from dict)
                                                          # plot that continent's year&life exp data
                                color=colors[continent],**kwargs) # in its color from dict; also accept kwargs
    ax[numbers[continent]].set(title=continent) # giving each  subplot the correct continent title
    ax[0].set(ylabel="Life Expectancy (years)") # labelling just the first y-axis


plt.tight_layout() # making things look nice

gapminder.groupby(["country"]).apply(f,alpha=.3) # running the above function, but grouping by country
                                                 # so each country has its own line on the plot
```

Out[587]: —



Problem 2

In this problem, you will create several attractive visualizations of the `gapminder` data set using the Seaborn package. Seaborn is a high-level interface to Matplotlib specially designed for working with Pandas data frames. Seaborn makes it relatively easy to create complex, multifaceted data graphics. The drawback is that it can sometimes be more difficult to exercise control over figure details, such as axis labels or figure size.

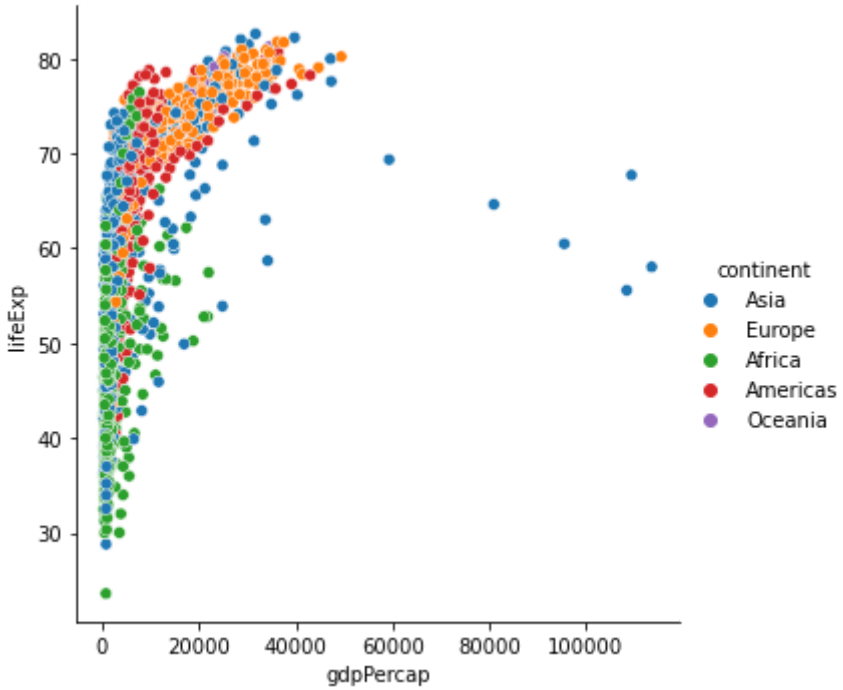
Part of the learning objective for this problem is for you to **practice learning from documentation and examples**. I have given you some pointers in each problem, but it will be up to you to fill in the blanks.

Import the `seaborn` package with the line `import seaborn as sns`. Then, run the code block below to create a scatterplot of `gdpPercap` (gross domestic product per capita) against `lifeExp` (life expectancy), with different colors for each continent.

In [588…

```
from matplotlib import pyplot as plt
import seaborn as sns

fgrid = sns.relplot(x = "gdpPercap",
                    y = "lifeExp",
                    hue = "continent",
                    data = gapminder)
```



The result of the above code is that `fgrid` is now a `FacetGrid` object (we'll see why it's called that in a minute). However, under the hood, we are still dealing with Matplotlib. To retrieve the individual plotting axis, we can access it from the `fgrid.axes` attribute, which is a 2d Numpy array containing the axes. For example, to make the horizontal axis logarithmic and labels, we can do this:

In [589…

```
fgrid = sns.relplot(x = "gdpPercap",
                    y = "lifeExp",
                    hue = "continent",
                    data = gapminder)

fgrid.axes[0][0].semilogx()

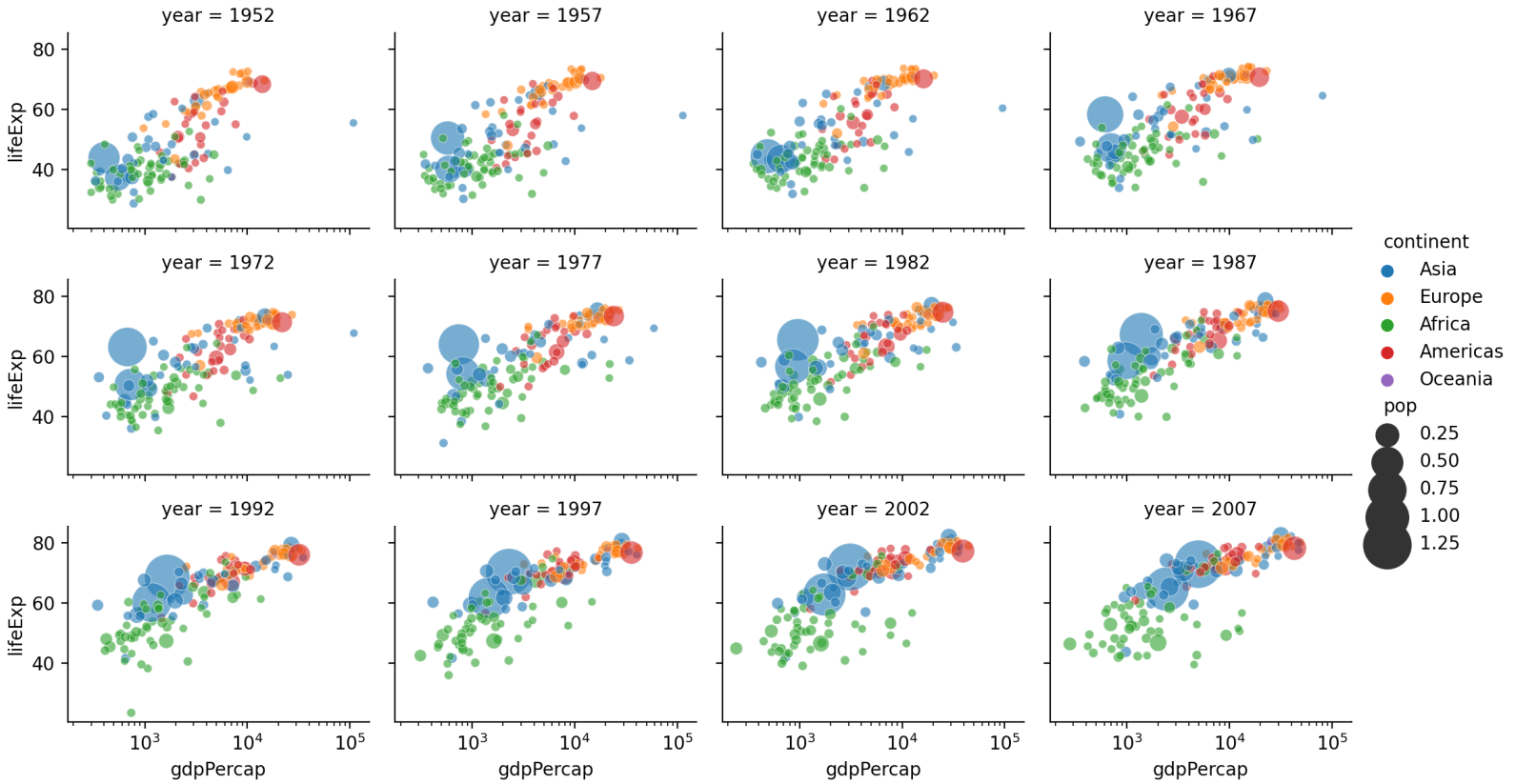
labels = fgrid.axes[0][0].set(xlabel = "GDP per capita",
                              ylabel = "Life Expectancy",
                              title  = "Impact of Economic Output on Life Expectancy")
```



Not bad! Note that seaborn has effectively looped over the different continents, without us needing to use `for`-loops or `apply`.

Part A

Create the following plot using Seaborn:



In this figure, each point represents a country, and the size of the point corresponds to the population.

In this part and the next, it is not necessary that your plot *precisely* match mine -- feel free to modify the sizes of labels, change the color scheme, etc.

Hints

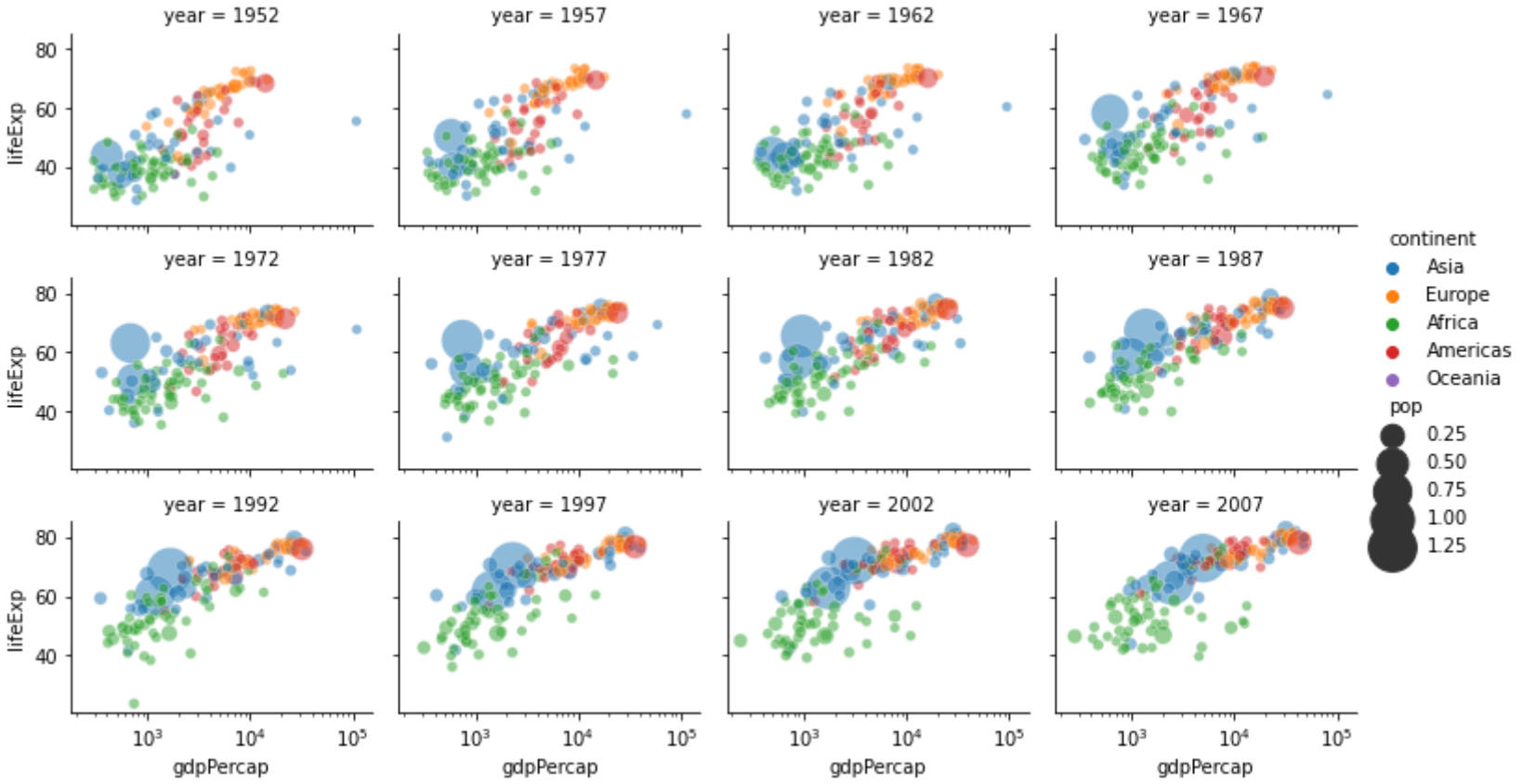
- You should use `sns.relplot` to create the figure.
- Look in the Seaborn Gallery for examples illustrating how to create a multi-panel figure without using `plt.subplots()`.
- The size of the figure is controlled by the `height` and `aspect` ratio arguments to `relplot`. I used `height = 2` and `aspect = 1.3`, but other choices are also fine.

- When using column wrapping to create the `3 x 4` axis array, the resulting `fgrid.axes` array is no longer 2-dimensional, but 1-dimensional.

In [590…

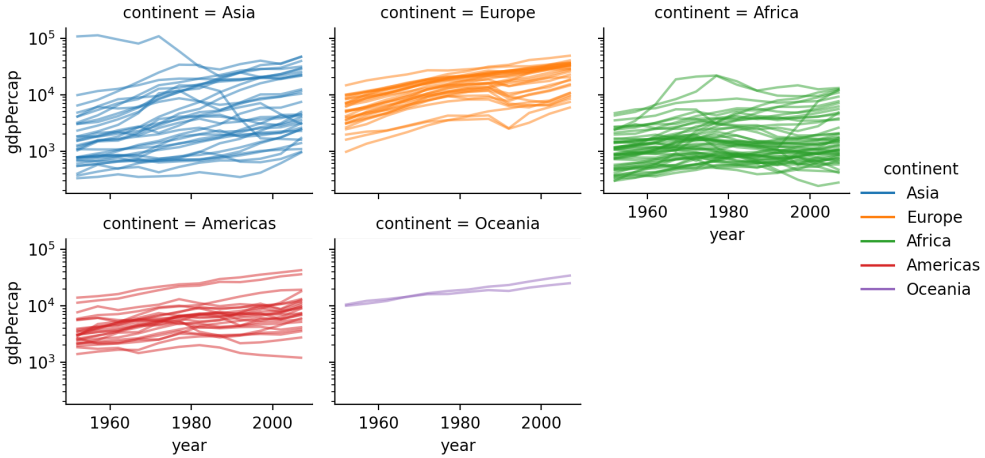
```
fgrid=sns.relplot(data=gapminder,
                  x="gdpPercap",
                  y="lifeExp",
                  hue="continent",
                  alpha=.5,
                  col="year",
                  aspect = 1.3,
                  height = 2,
                  col_wrap=4,
                  size="pop",
                  sizes=(29,700))

for ax in fgrid.axes:
    ax.semilogx()
```



Part B

Create the following plot using Seaborn:



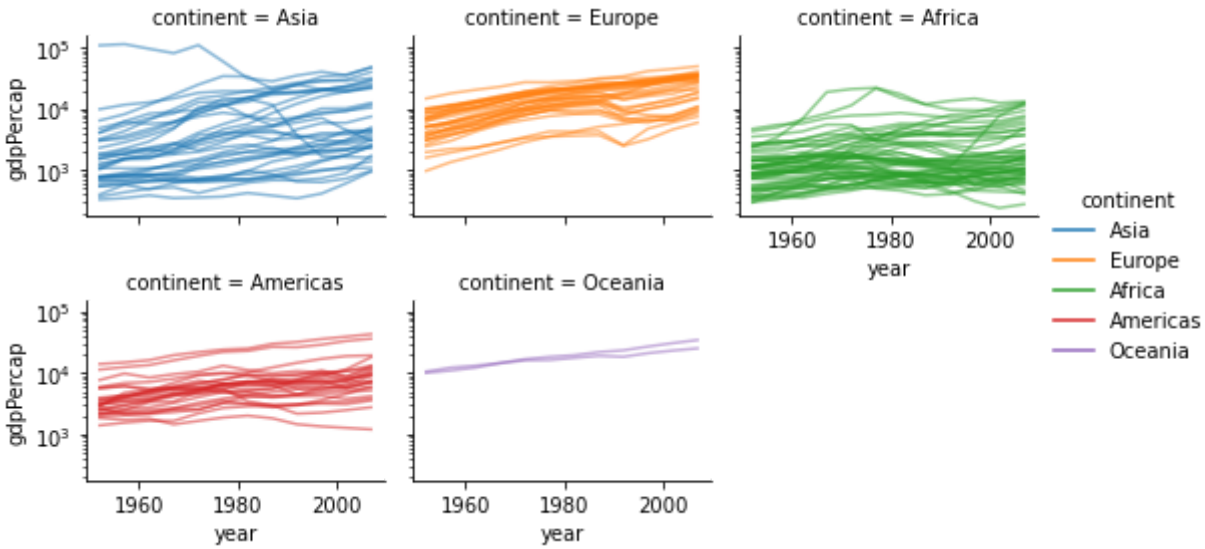
Hints

- The easiest way is to again use `relplot`. This function defaults to producing scatterplots, but if you check the gallery you'll find examples of how to specify keywords that create line charts instead.

In [591…

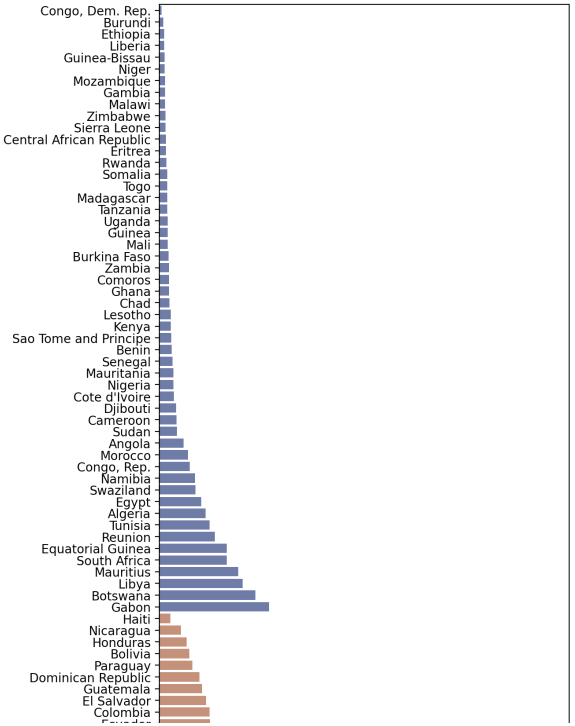
```
fgrid=sns.relplot(data=gapminder,
                  x="year",
                  y="gdpPercap",
                  hue="continent",
                  alpha=.5,
                  col="continent",
                  aspect = 1.3,
                  height = 2,
                  kind="line",
                  col_wrap=3,
                  units=gapminder[("country")],
                  estimator=None)

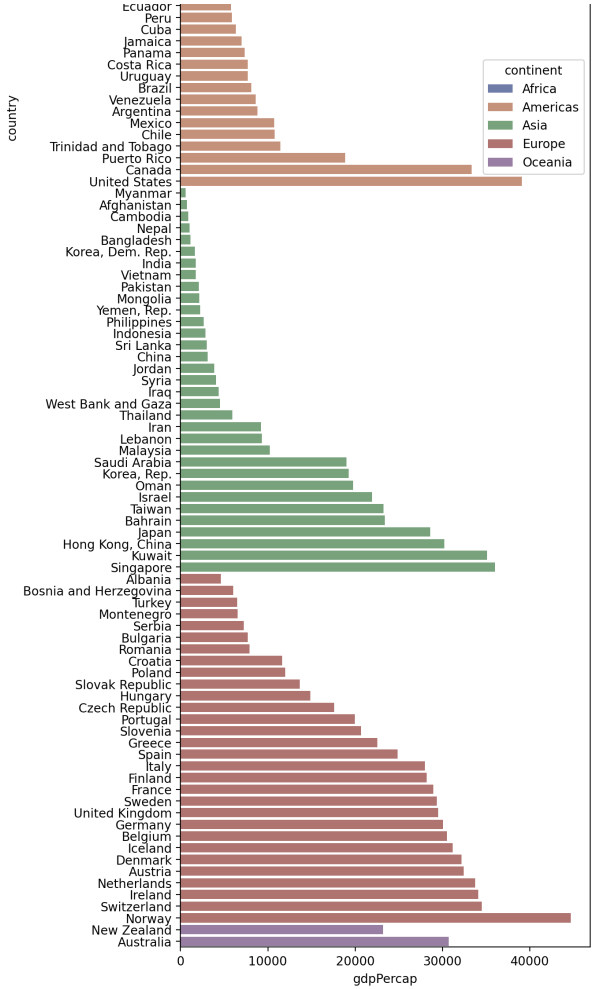
for ax in fgrid.axes:
    ax.semilogy()
```



Part C

Create the following (very long) plot using Seaborn:





Hints

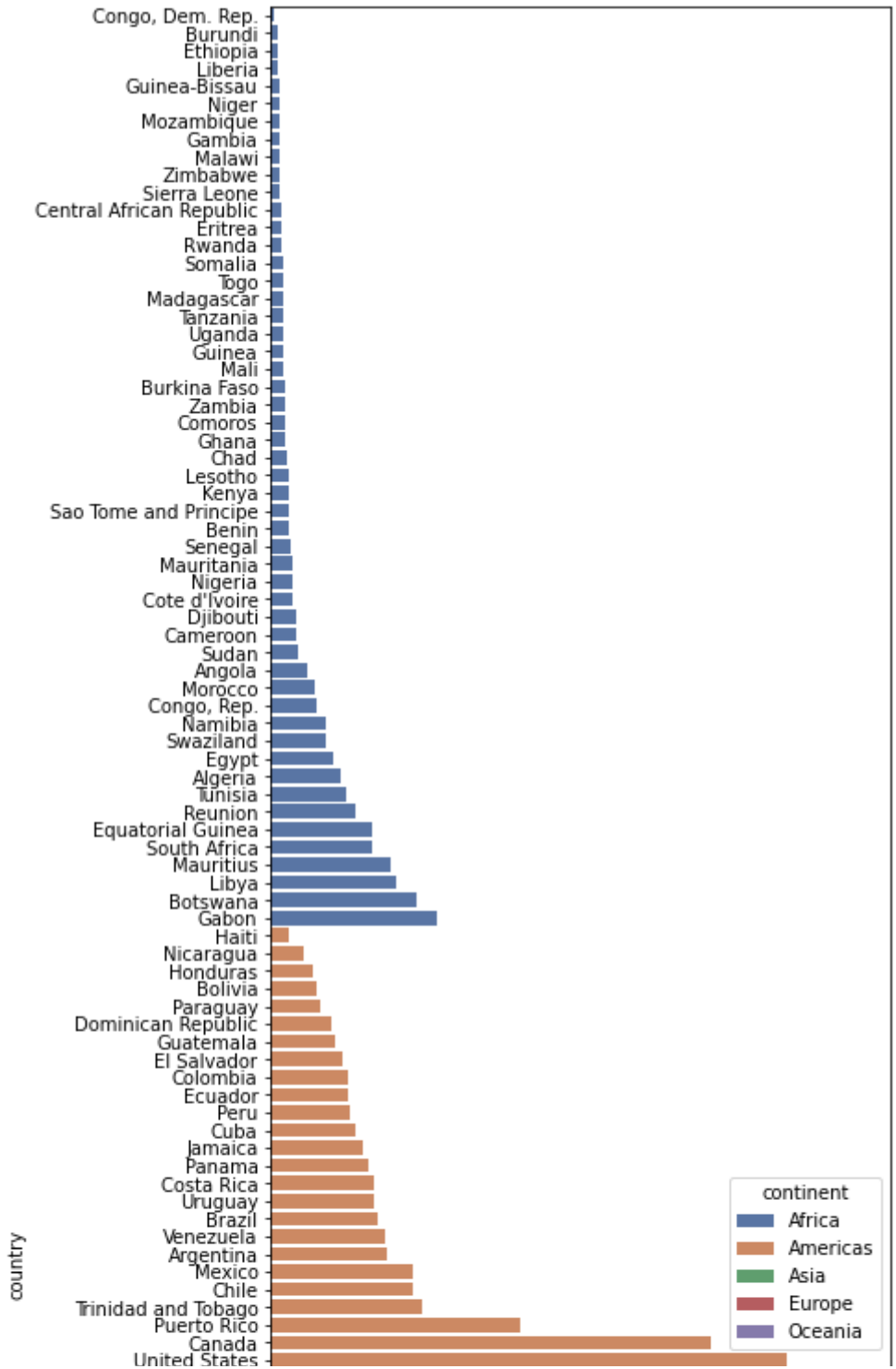
- Use `sns.barplot`.
- Pick a single year of the `gapminder` data frame to show. In the example, I showed data from 2002 only.
- To sort the bars in ascending order by continent, you should sort the data frame that you pass to `sns.barplot`.
- The argument `dodge = False` is a helpful keyword that will keep your bars centered.
- I used a different color palette just to shake things up a little.

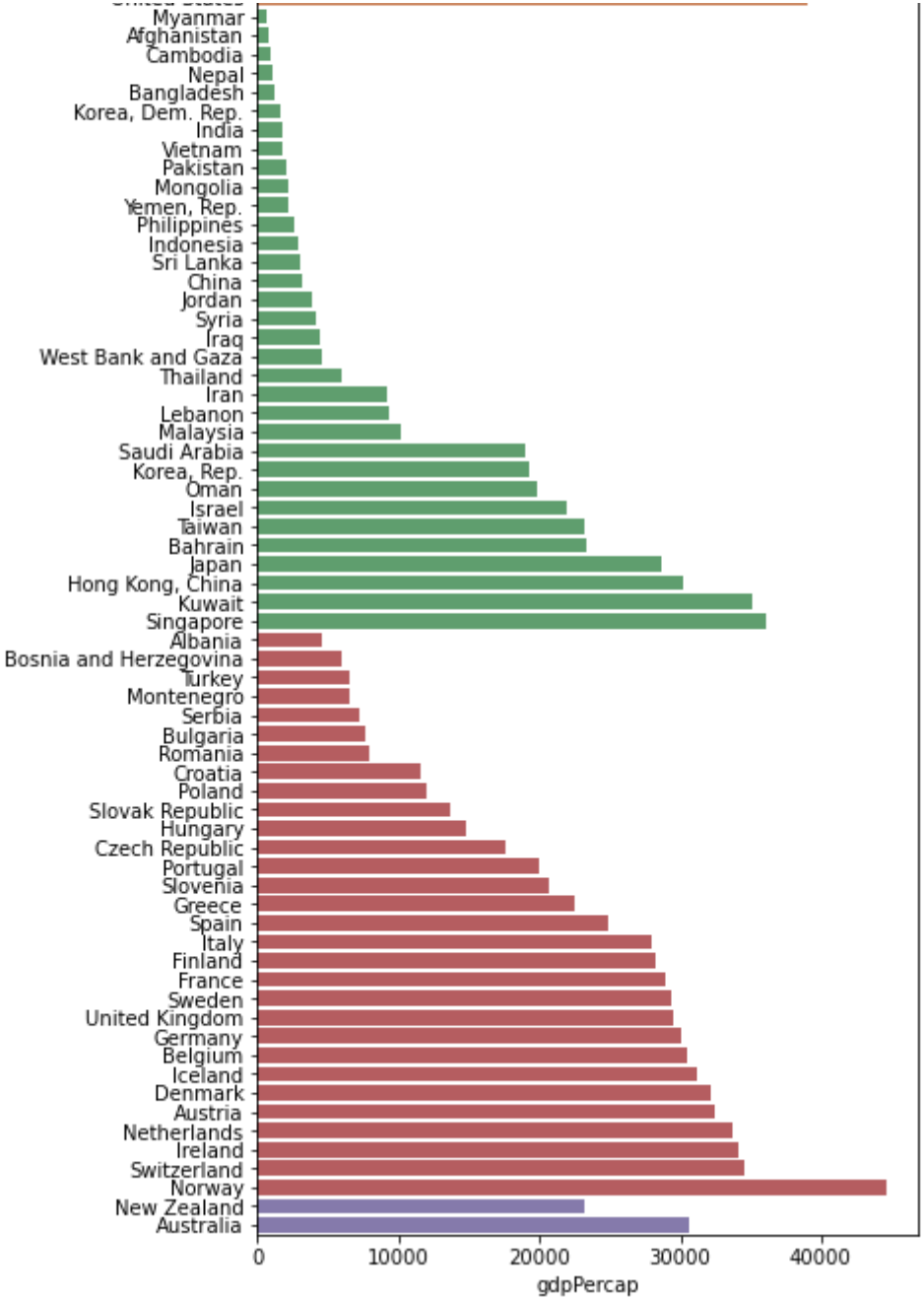
In [592…

```
fig,ax=plt.subplots(figsize=(6,25))
sub_gapminder=gapminder[gapminder["year"]==2002]
df=sub_gapminder
```

```
df=df.sort_values(["continent","gdpPercap"],ascending=True)
sns.barplot(data=df,
            x="gdpPercap",
            y="country",
            hue="continent",
            dodge=False,
            palette="deep")
```

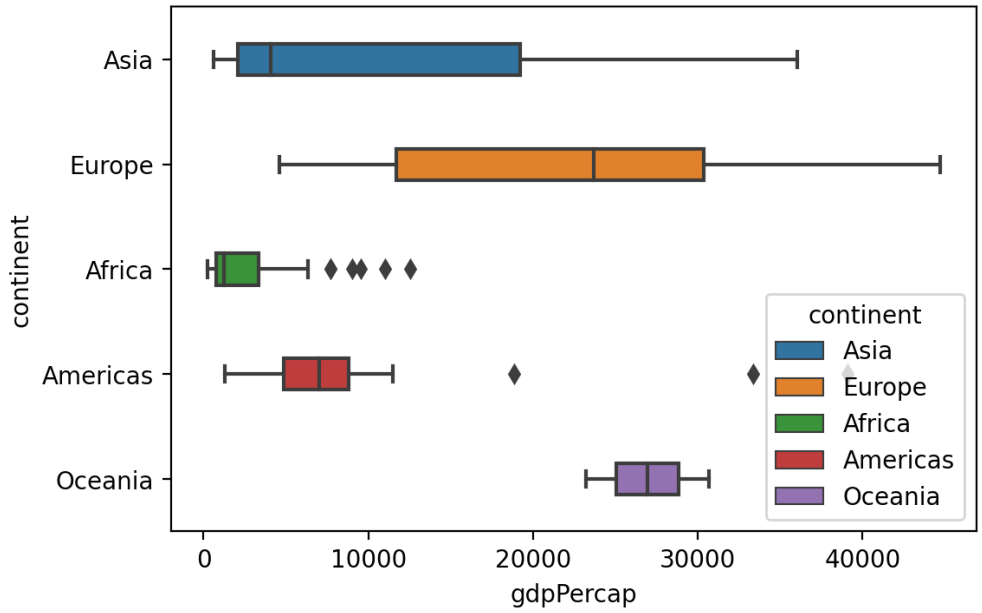
Out[592]: <AxesSubplot:xlabel='gdpPercap', ylabel='country'>


Congo, Dem. Rep.
Burundi
Ethiopia
Liberia
Guinea-Bissau
Niger
Mozambique
Gambia
Malawi
Zimbabwe
Sierra Leone
Central African Republic
Eritrea
Rwanda
Somalia
Togo
Madagascar
Tanzania
Uganda
Guinea
Mali
Burkina Faso
Zambia
Comoros
Ghana
Chad
Lesotho
Kenya
Sao Tome and Principe
Benin
Senegal
Mauritania
Nigeria
Cote d'Ivoire
Djibouti
Cameroon
Sudan
Angola
Morocco
Congo, Rep.
Namibia
Swaziland
Egypt
Algeria
Tunisia
Reunion
Equatorial Guinea
South Africa
Mauritius
Libya
Botswana
Gabon
Haiti
Nicaragua
Honduras
Bolivia
Paraguay
Dominican Republic
Guatemala
El Salvador
Colombia
Ecuador
Peru
Cuba
Jamaica
Panama
Costa Rica
Uruguay
Brazil
Venezuela
Argentina
Mexico
Chile
Trinidad and Tobago
Puerto Rico
Canada
United States
country
continent
Africa
Americas
Asia
Europe
Oceania


Myanmar
Afghanistan
Cambodia
Nepal
Bangladesh
Korea, Dem. Rep.
India
Vietnam
Pakistan
Mongolia
Yemen, Rep.
Philippines
Indonesia
Sri Lanka
China
Jordan
Syria
Iraq
West Bank and Gaza
Thailand
Iran
Lebanon
Malaysia
Saudi Arabia
Korea, Rep.
Oman
Israel
Taiwan
Bahrain
Japan
Hong Kong, China
Kuwait
Singapore
Albania
Bosnia and Herzegovina
Turkey
Montenegro
Serbia
Bulgaria
Romania
Croatia
Poland
Slovak Republic
Hungary
Czech Republic
Portugal
Slovenia
Greece
Spain
Italy
Finland
France
Sweden
United Kingdom
Germany
Belgium
Iceland
Denmark
Austria
Netherlands
Ireland
Switzerland
Norway
New Zealand
Australia
0
10000
20000
30000
40000
gdpPercap

Part D

Create the one last plot using Seaborn:

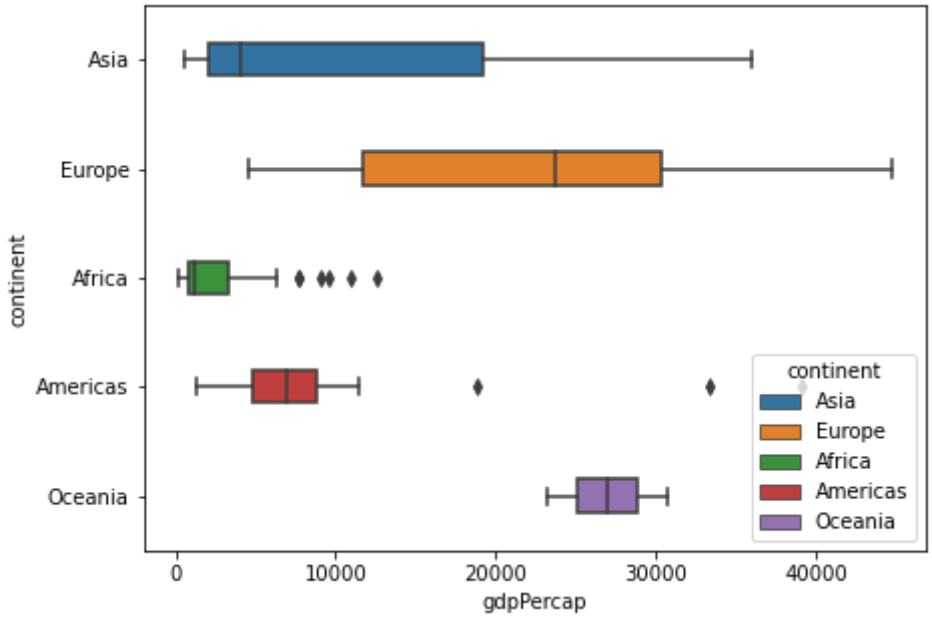


Hints

- This is a boxplot.
- You should again pick a single year of the `gapminder` data frame to show. In the example, I show data from 2002 only.

In [593…

```
fig,ax=plt.subplots(figsize=(7,5))
sub_gapminder=gapminder[gapminder["year"]==2002]
df=sub_gapminder
sns.boxplot(data=df,
            x="gdpPercap",
            y="continent",
            hue="continent",
            dodge=False,
            width=0.3)
sns.move_legend(ax, "lower right")
```



Problem 3

In our first lecture on machine learning, we did linear regression "by hand." In this problem, we will similarly perform logistic regression "by hand." This homework problem is closely parallel to the lecture, and so you might want to have the notes handy.

Although logistic regression is a relatively simple model, it is a foundation for many modern deep neural nets. Additioanlly, logistic regression itself is still often used in applied contexts even now.

> "When we raise money it's AI, when we hire it's machine learning, and when we do the work it's logistic regression."
>
> (I'm not sure who came up with this but it's a gem 💎)
>
> — Dr. Daniela Witten (@daniela_witten) September 26, 2019

Whereas linear regression is often used to measure arbitrary quantities like GDP or happiness scores, logistic regression is primarily used to estimate *probabilities*. For example, we might use logistic regression to estimate the probability of a passenger surviving the Titanic crash, a borrower defaulting on a loan, or an email being spam.

For concreteness, let's say that we are considering the latter case. Suppose that we wish to model the probability that an email is spam as a function of the proportion of flag words (like "investment", "capital", "bank", "account", etc.) in the email's body text. Call this proportion x. x is then a variable between 0 and 1.

In logistic regression, we suppose that the probability p that an email is spam has the form

$$p = \frac{1}{1 + e^{-ax-b}} \ ,$$

where a and b are again parameters. Let's see how this looks.

In [594…

```
# run this block

import numpy as np
from matplotlib import pyplot as plt

n_points = 1000

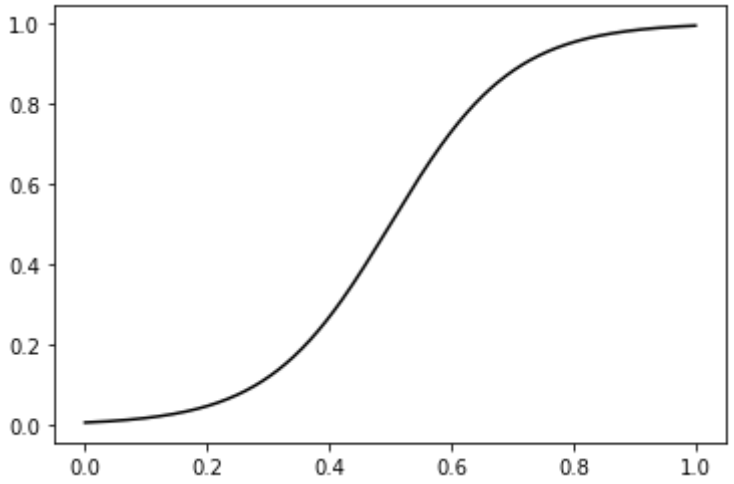
a = 10
b = -5

x = np.sort(np.random.rand(n_points))
p = 1/(1+np.exp(-a*x - b))

fig, ax = plt.subplots(1)
ax.plot(x, p, color = "black")
```

Out[594]:

```
[<matplotlib.lines.Line2D at 0x7fae79d56160>]
```



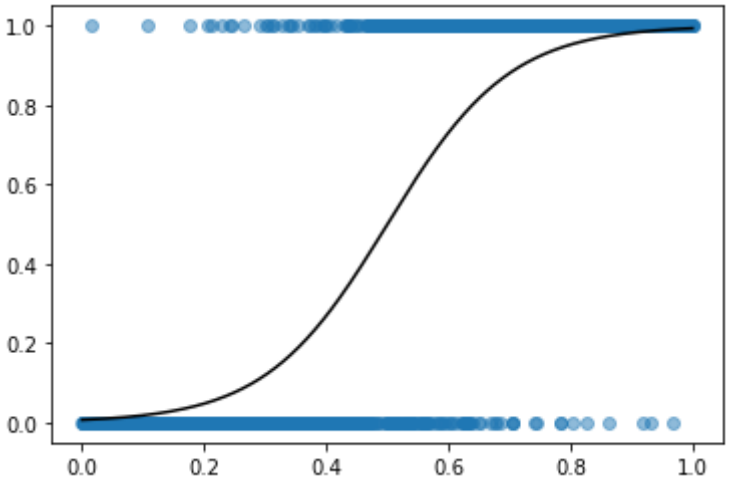
As usual, in practice we don't have access to the true function telling us the probability that an email is spam. Instead, we have access to data telling us whether or not the email really IS spam. We can model this situation by flipping a biased coin for each email, with the probability of heads determined by the modeled probability.

In [595…
```
# run this block
y = 1.0*(np.random.rand(n_points) < p)
```

A value of 1 indicates that the email is indeed spam, while a value of 0 indicates that the email is not spam.

In [596…
```
# run this block
ax.scatter(x, y,  alpha = 0.5)
fig
```

Out[596]:



Notice that there are more spam emails where the model gives a high probability, and fewer where the model gives a lower probability. However, there may be some non-spam emails with even high probability -- sometimes we get legitimate emails about bank accounts, investments, etc.

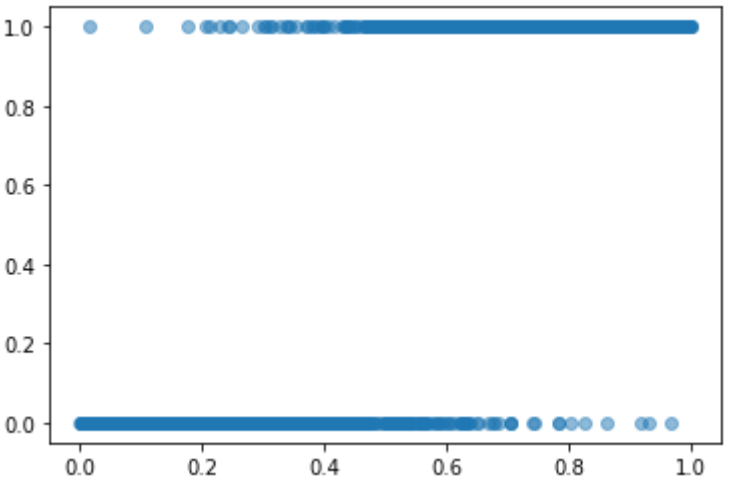
Of course, we don't have access to the true model, so our practical situation looks more like this:

In [597…

```
# run this block
fig, ax = plt.subplots(1)
ax.scatter(x, y, alpha = 0.5)
```

Out[597]:

```
<matplotlib.collections.PathCollection at 0x7fae7ae83760>
```



We would like to use logistic regression to try to recover something close to the true model.

Part A

Write the model function `f`. The arguments of `f` should be the predictor variables `x` and the parameters `a` and `b`. The output of `f` should be the spam probabilities under the logistic model (see equation above) for these data. Use `numpy` tools, without `for`-loops. If you scan the above code carefully, you'll observe that most of this code is already written for you.

This is a simple function, but **please add a short docstring indicating** what kinds of input it accepts and how to interpret the output.

Comments are necessary only if your function body exceeds one line.

In [598…

```
# your solution
import numpy as np

def f(x,a,b):
    """
    Function predicts the probability that an email is spam using a logistic regression equation.

    parameter 1, x: a predictor variable, used to predict the probability
    parameter 2, a: a parameter (coefficient variable) in the probability equation
    parameter 3, b: a parameter (coefficient variable) in the probability equation

    returns the probability of the email being spam using a logisitic model/equation
    """
    return (1/(1+np.exp(-a*x - b)))
```

Part B

Plot 10 candidate models against the data, using randomly chosen values of `a` between 5 and 15 and randomly chosen values of `b` between -2.5 and -7.5. Your plot should resemble in certain respects the third plot in these lecture notes.

Comments are not necessary in this part.

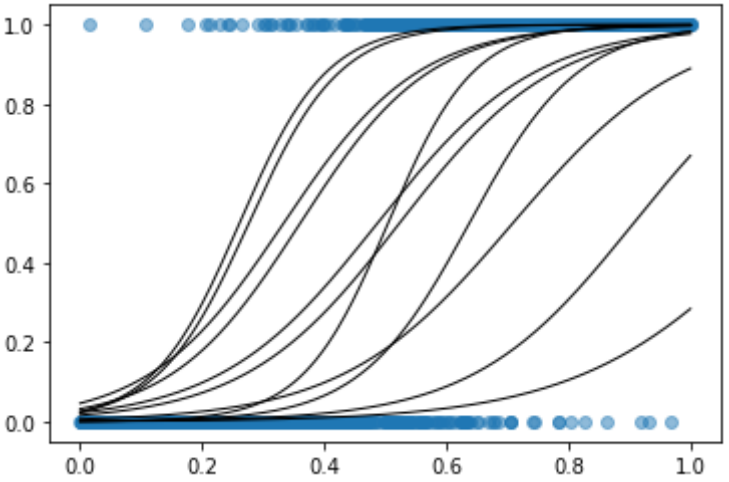
In [605…

```
# your solution here
fig,ax=plt.subplots(1)
for i in range(11):
```

```
    a = 10*np.random.rand()+5
    b = -5*np.random.rand()-2.5

    ax.plot(x,f(x,a,b),color= "black",linewidth=1)
ax.scatter(x,y,alpha = 0.5)
```

Out[605]: <matplotlib.collections.PathCollection at 0x7fae30643070>



Part C

The *loss function* most commonly used in logistic regression is the *negative cross-entropy*. The negative cross-entropy of the `i` th observation is

$$-\left[y_i \log \hat{p}_i + (1-y_i)\log(1-\hat{p}_i)\right]$$

where $y_i \in \{0,1\}$ is the `i` th entry of the target data and $\hat{p}_i$ is the model's estimated probability that $y_i = 1$. The negative cross-entropy of the entire data set is the sum of the negative cross-entropies for each individual observation.

Write a function that computes the negative cross entropy as a function of `x`, `y`, `a`, and `b`. This can be done in no more than two lines using `numpy`, without `for`-loops. Don't forget which logarithm is #BestLogarithm.

As in Part B, please write a short docstring describing what your function does and what inputs it accepts. Comments are necessary only if your function body exceeds two lines.

In [606…

```
# your solution here
```

```
def neg_entropy(x,y,a,b):
    """
    A function that calculates the negative cross-entropy of a dataset by
    summing the negative cross-entropy for each individual observation.
    Uses a loss function.

    parameter 1, x: a predictor variable, used to predict the probability
    parameter 2, y: the actual target data entry rather than the probability
    parameter 3, a: a parameter (coefficient variable) in the probability equation
    parameter 4, b: a parameter (coefficient variable) in the probability equation

    returns the the negative cross-entropy of the entire dataset
    """
    preds = f(x,a,b)
    return np.sum(-(y*np.log(preds)+(1-y)*np.log(1-preds)))
```

Part D

On a single axis, plot 100 distinct models (using the code that you wrote in) in Part B. Highlight the one with the lowest negative cross entropy in a different color -- say, red. Compare the best values of `a` and `b` that you found to the true values, which were `a = 10` and `b = -5`. Are you close?

The plot you produce should resemble, in some respects, the fifth plot in these lecture notes.

It is not necessary to write comments in this part.

In [607…

```
# your solution here

fig,ax=plt.subplots(1)
best_a, best_b = 0,0
best_loss = np.inf

for i in range(100):
    a = random.randint(5,10)
    b = random.uniform(-7.5,-2.5)

    ax.plot(x,f(x,a,b),
            color= "black",
            linewidth=1,
            alpha=.2)
```

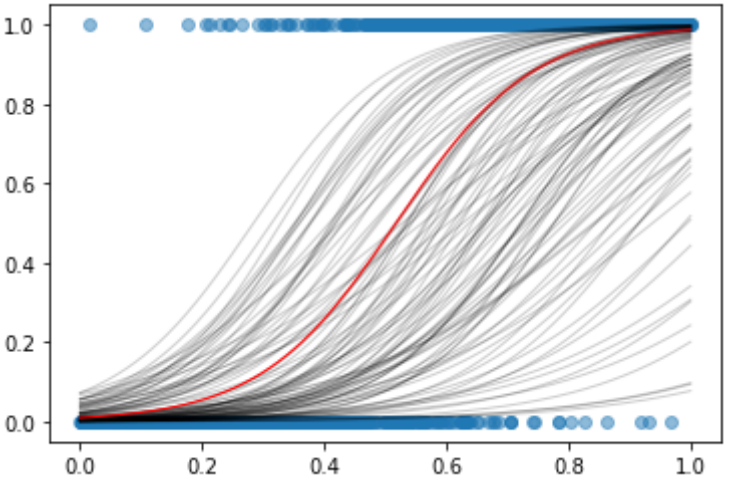
```
    loss = neg_entropy(x,y,a,b)
    if loss < best_loss:
        best_loss=loss
        best_a,best_b=a,b

ax.scatter(x,y,alpha = 0.5)


ax.plot(x,f(x,best_a,best_b),
    color= "red",
    linewidth=1)

print(best_a,best_b) #pretty close to actual values of 10 and -5
```

```
9 -4.654640129985651
```



In [ ]:

It is not required to use `scipy.optimize` to more accurately estimate `a` and `b` for this homework assignment, but you are free to do so if you wish. You may then use the optimal estimates in the following part.

Part E

In classification tasks, we evaluate not just the standard loss function, but also the *accuracy* -- how often does the model correctly classify the data? Let's say that the model classifies an email as spam according to the following rule:

1. If $\hat{p}_i$ (the model probability plotted above) is larger than c, classify the email as spam.

2. If $\hat{p}_i$ is less than or equal to c, classify the email as not-spam.

Write a function called `positive_rates` which accepts the following arguments:

1. The data, `x` and `y`.
2. The best parameters `best_a` and `best_b`.
3. A threshold `c` between 0 and 1.

This function should output two numbers. The first of these is *false positive rate*: the proportion of non-spam emails that the model incorrectly labels as spam. The second is the *true positive rate*: the proportion of spam emails that the model correctly labels as spam.

For example:

```
positive_rates(x, y, best_a, best_b, c = 0.5)
```

```
(0.05058365758754864, 0.7469135802469136)
```

Note: due to randomization, your numerical output may be different.

Please write a descriptive docstring for your function. Comments are necessary only if your function body exceeds five lines.

In [608…

```
# your solution here
def positive_rates(x,y,best_a,best_b,c):
    """
    Calculates false positive rate (proportion of non-spam emails that model labels as spam)
    and true positive rate (proportion of spam emails that the model correctly labels as spam).

    parameter 1, x: data, predictor variables used to predict probabilities
    parameter 2, y: data, email probabilities
    parameter 3, best_a: best parameter a that minimizes loss most, as calculated by part D
    parameter 4, best_b: best parameter b that minimizes loss most, as calculated by part D
    parameter 5, c: threshold between 0 and 1, specified by user,
                    probability above c means email is spam,
                    probability equal or below c is not spam

    Returns tuple of false positive rate and true positive rate.
    """
    preds = f(x,best_a,best_b) # creating array of prediction probabilities using data and best a/b
    preds[preds>c] = 1 # probability prediction above threshold is spam
    preds[preds<=c]= 0 # probability prediction below or equal to threshold is not spam
```

```
    false=0 # to keep track of false positives
    true=0  # to keep track of true positives

    for i in range(preds.size): # looping through preds
        if preds[i]!=y[i] and preds[i]==1: # if the prediction does not match the target,
                                           # and pred is spam
            false+=1                       # this is a false positive, increase false count by 1
        if preds[i]==y[i] and preds[i]==1: # if the prediction matches the target,
                                           # and pred is spam
            true+=1                        # this is a true positive, increase true count by 1

    non_spam=y[y==0]                       # making an array of actual non-spam emails
    total_ns = non_spam.size               # counting total number of NS emails
    spam=y[y==1]                           # making an array of actual spam emails
    total_s = spam.size                    # counting total number of spam emails

    return(false/total_ns,true/total_s) # returning false positive rate (false positive/total non-spam)
                                        # and true positive rate (true positive/total spam)
```

In [609…

```
# demonstrate your function here

positive_rates(x, y, best_a, best_b, c = 0.5)
```

Out[609]:

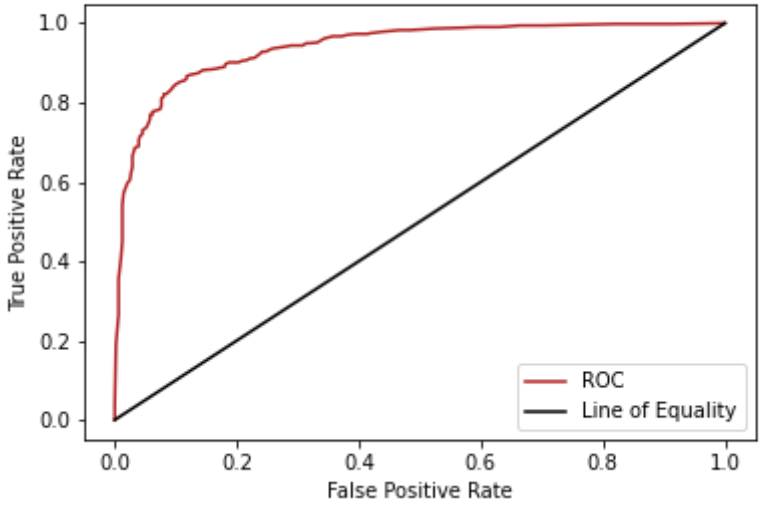
```
(0.13238289205702647, 0.8467583497053045)
```

Part F

Plot the *receiver operating characteristic* (ROC) curve for the logistic model with parameters `best_a` and `best_b`. The ROC curve is the plot of the `false_positive` rate (on the horizontal axis) against the `true_positive` rate (on the vertical axis) as the threshold `c` is allowed to vary. Additionally, plot a diagonal line ("the line of equality") between the points (0,0) and (1,1). Your ROC curve should lie noticeably above the line of equality. Plot your curves in different colors and add a legend to help your reader understand the plot.

It is ok to use `for`-loops and list comprehensions in this part.

Your result should look something like this, although it won't be exactly the same due to randomness.



In [615…

```
# plot code here
fig,ax=plt.subplots(1) # creating a single plot to use

c_values = np.linspace(0, 1, 100)  # creating array of 100 evenly spaced points between 0 and 1
                                   # to use as thresholds later
fp=[]                              # empty false positive list
tp=[]                              # empty true positive list

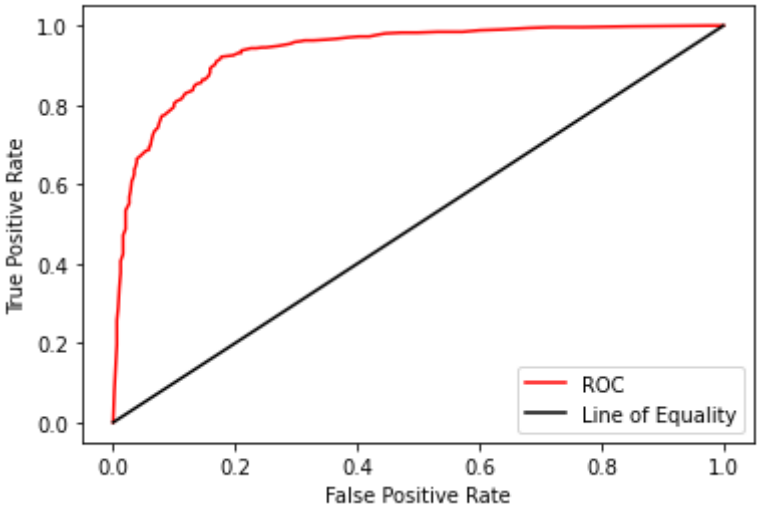

for i in c_values:   # looping through the c_value array
    fp_new,tp_new = positive_rates(x, y, best_a, best_b, c = i) # running the rate function for each
                                                               # threshold or c_value
    fp.append(fp_new) # append the false positive rate to list
    tp.append(tp_new) # append the true positive rate to list

# plot fp rate list (x-axis) vs tp rate list (y-axis) in red
# label curve
ax.plot(fp,tp,color = "red",label="ROC")

plt.plot((0,1),(0,1), color = "black",label="Line of Equality") # plot line from (0,0) to (1,1) in black

ax.set(xlabel="False Positive Rate", ylabel="True Positive Rate") # label axes
ax.legend() # adding legend
```

Out[615]: <matplotlib.legend.Legend at 0x7fae65629850>



Generally speaking, a "good" classifier is one that can reach the closest to the top-left corner of the ROC diagram. This is a classifier that can achieve a high rate of true positives, while keeping a low rate of false positives. There are various ways to measure how "good" an ROC curve is, which are beyond our present scope.